EduAirControl: An IoT-Enabled Microservices Platform for Environmental Monitoring and Analysis of Educational Spaces

Jhon Jairo Cortez Vargas · María de los Ángeles Olaya García
Technologist in Software Analysis and Development, Group (Ficha) 3145556
Servicio Nacional de Aprendizaje (SENA) Industria, Neiva, Huila, Colombia
Instructor: Karol Daniela Correa

Abstract

Educational institutions rarely have a single place to check the environmental conditions of their classrooms: temperature, humidity, air quality and noise are usually measured in isolation, or not measured at all. This paper describes EduAirControl, a web and mobile platform that centralizes those measurements, links each one to the educational environment and the sensor that produced it, and lets users view them in real time and analyze them by day, week, month and year. The system was built with seven backend modules (an API Gateway and six microservices) in Java 17 and Spring Boot 4, with OAuth2/JWT authentication, PostgreSQL persistence managed with Liquibase, RabbitMQ messaging, Redis caching, a React frontend and an Expo mobile app. A prototype with an ESP32 and DHT11, KY-038 and MQ-135 sensors validates physical capture. Quality was addressed with an eight-level test plan, 48 functional cases and 285 automated test methods. The results reported are those that exist at the time of writing: the full execution of the plan has not been closed, the final sending firmware and the MQ-135 calibration are pending, and a code audit left 11 active risks, two of them critical. The main contribution is a documented, verifiable reference architecture for environmental monitoring in educational settings.

Keywords: environmental monitoring, Internet of Things, microservices, hexagonal architecture, Spring Boot, ESP32, educational spaces

Introduction

The Problem

The environmental conditions of a classroom affect the comfort and performance of the people who use it. The review by Mendell and Heath (2005) links indoor air pollutants and thermal conditions in schools to student performance, and Shield and Dockrell (2003) document the effects of noise on children in school settings. Even so, the data produced by sensors tends to be scattered: each device shows its own value, nobody keeps the history, and there is no clear link between a measurement and the space where it was taken.

Whoever manages the spaces cannot compare how conditions change over time or make decisions based on data. Five dimensions of the problem follow: access (there is no clear place to consult the information), centralization (measurements stay detached from the space), understanding (raw values do not explain how conditions evolve), traceability (a measurement is only useful if its origin and time are known) and governance (who views and who administers must be controlled).

Research Motivation

The motivation was to build a system that not only worked but was understandable, measurable and maintainable, and that could serve as a case study of a SENA training project in which documentation, design decisions and tests matter as much as code. It was also meant to show that a low-cost IoT prototype can be integrated with a professional-grade microservices architecture.

Research Objectives

The general objective was to develop a platform that makes it possible to monitor, manage and analyze the environmental conditions of educational spaces using IoT sensor data. The specific objectives were:

- Implement authentication and authorization with four roles (SUPER_ADMIN, ADMIN, USER and VIEWER).
- Manage campuses, educational environments, sensors and devices.
- Receive, validate, store and analyze measurements, and raise alerts based on thresholds.
- Provide web and mobile interfaces with a dashboard, favorites, preferences, languages and accessibility.
- Validate physical data capture with an ESP32-based prototype.
- Document and test the system so that its results can be verified.

Importance of the Topic

The Internet of Things connects physical objects with software services so that their surroundings can be observed continuously (Atzori et al., 2010; Gubbi et al., 2013). In educational spaces, that capability enables data-based decisions about ventilation, noise or room use. EduAirControl is a practical application case in a Colombian setting. The product does not promise automatic physical control or predictive management; those capabilities are outside the current scope and are proposed as future work.

Related Work

This section is not a systematic review: it gathers the references that support the project's technical decisions on four fronts.

Internet of Things and Environmental Monitoring

Atzori et al. (2010) describe the Internet of Things as the convergence of object-oriented, internet-oriented and semantics-oriented visions, and Gubbi et al. (2013) propose a reference architecture in which sensors feed storage, analytics and visualization services. EduAirControl adopts that idea: the sensor captures, the backend validates and stores, and the interfaces present.

Microservices and Hexagonal Architecture

Fowler and Lewis (2014) define microservices as small independent services that communicate through lightweight mechanisms and are deployed separately. Newman (2015) and Richardson (2018) describe their trade-offs, including data consistency and asynchronous communication. To organize the inside of each service, hexagonal architecture was used (Cockburn, 2005), which separates the domain from inbound and outbound adapters, in line with the principles of domain-driven design (Evans, 2003).

API Security

The OAuth 2.0 framework (Hardt, 2012) and JWT tokens (Jones et al., 2015) are the basis of the system's authentication. The OWASP Top 10 list (OWASP Foundation, 2021) was used as a reference to review access controls, security configuration and exposure of secrets.

Software Quality and Testing

ISO/IEC 25010 (International Organization for Standardization & International Electrotechnical Commission [ISO/IEC], 2011) defines quality characteristics such as reliability, security and maintainability. Martin (2008) argues that clean code lowers maintenance costs, and Forsgren et al. (2018) link continuous delivery practices to team performance. Nygard (2018) presents the circuit breaker pattern to limit cascading failures between services, a pattern the project documented but has not yet implemented.

Methodology

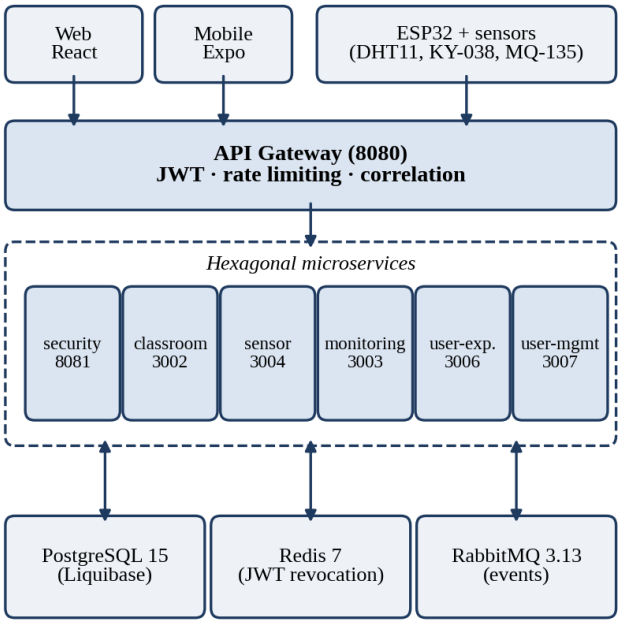
Specification-Driven Development

The team worked with specification-driven development: context, domain, product and requirements were documented first, followed by architecture, data and API contracts. The documentation was organized into 15 folders covering governance, context, domain, product, requirements, architecture, data, API, UML, microservices, DevOps, quality, operations and project control. Design decisions were recorded as 17 architecture decision records (ADRs), and service contracts were described with OpenAPI (OpenAPI Initiative, 2021), with 53 documented paths.

System Architecture

The system follows a client–server model, as Figure 1 shows. The web and mobile applications consume an HTTP/JSON API exposed by an API Gateway (Spring Cloud Gateway, port 8080) that routes requests, validates the JWT, limits requests and propagates a correlation ID. Six microservices sit behind it, each with its own database.

Figure 1
EduAirControl General Architecture



Note. Ports correspond to the local configuration. Authors' own work.

Technology Selection

Table 1 summarizes the technology stack. Technologies widely used in industry and with abundant documentation were chosen, because the project is developed in a training context.

Table 1
EduAirControl Technology Stack

Layer	Technology
Backend	Java 17, Spring Boot 4.0.5, Spring Data JPA
Security	Spring Authorization Server, RS256 JWT, Google and Facebook sign-in
Data	PostgreSQL 15, Liquibase
Messaging and cache	RabbitMQ 3.13, Redis 7
Web	React 19, Vite 8, React Router, i18next, Recharts
Mobile	Expo 54, React Native 0.81, AsyncStorage, BLE PLX
Deployment	Docker Compose, GitHub Actions continuous integration, Kubernetes manifests

Functional Modules

Table 2 presents the backend modules and their responsibilities.

Table 2
Backend Modules

Service	Responsibility	Port
ms-security	Registration, sessions, OAuth2/OIDC, roles and institutions	8081
ms-classroom-management	Campuses, environment types and environments	3002
ms-environment-monitoring	Measurements, dashboard, analysis, thresholds and alerts	3003
ms-sensor-management	Catalog, sensors, installations and devices	3004
ms-user-experience	Favorites, preferences and ratings	3006
ms-user-management	User profile	3007

Note. The API Gateway listens on port 8080.

Security

User access is based on JWTs signed with RS256, with expiration and revocation: on logout the token is recorded in Redis and any reuse is rejected with a 401 code. Services validate the signature against the ms-security public key set (JWKS). The four roles separate administrative operations from read-only ones. Devices use an X-API-Key that is shown only once when it is created or rotated. Secrets are configured through environment variables and CORS is configurable.

IoT Prototype

The prototype uses an ESP32 as the controller, a DHT11 for temperature and humidity, a KY-038 for the relative sound level and an MQ-135 for the relative variation of gases (Espressif Systems, n.d.). The ESP32 is provisioned over BLE from the mobile app. The MQ-135 needs calibration and voltage adaptation before the analog converter, so it is not presented as a calibrated CO_2 meter in ppm.

Implementation

Structure of a Microservice

Each service is an independent Maven project organized by context, with domain, application and infrastructure packages. The domain holds the rules (for example, threshold evaluation and environmental analysis), the application layer orchestrates use cases, and the infrastructure layer contains controllers, persistence and event consumers.

Persistence and Migrations

Each service keeps its own schema, versioned with Liquibase. Hibernate is configured in validation mode (ddl-auto=validate), so the schema changes only through migrations and any drift stops startup.

Messaging and Consistency

Services communicate through events on RabbitMQ. To avoid losing events between the database write and publication, classroom and sensor installations use the outbox pattern (Richardson, 2018): the event is stored in the same transaction and a scheduled process publishes it. The monitoring service consumes the events of the other services.

Flow of a Measurement

The environmental information flow has four steps:

- Sensors capture the variables of the educational space.
- The ESP32 processes the readings and, in the final integration, sends them to the ingestion endpoint.
- The backend validates the X-API-Key, processes the measurement and stores it in PostgreSQL; events travel through RabbitMQ.
- The web and mobile apps query the API and display measurements, statuses, analyses and alerts.

Deployment

The local environment is started with Docker Compose and a wait script that checks that services are healthy. A smoke test walks the end-to-end flow (register, login, logout and reuse of a revoked token). For production, Kubernetes manifests with a secrets template were prepared.

Testing and Validation

Testing Strategy

The test plan (version 1.0) defines eight levels: unit, integration, contract, functional, system, IoT, regression and non-functional. It includes 48 functional cases: 12 for security, 4 for environments, 4 for sensors, 7 for monitoring, 7 for web, 6 for mobile and 8 for IoT. The techniques used are equivalence partitioning, boundary values, negative tests, role-based tests and black-box tests. Each case ends in one of five results (PASS, FAIL, BLOCKED, NOT RUN or NOT APPLICABLE), and a pending test is never recorded as passed.

Automated Tests

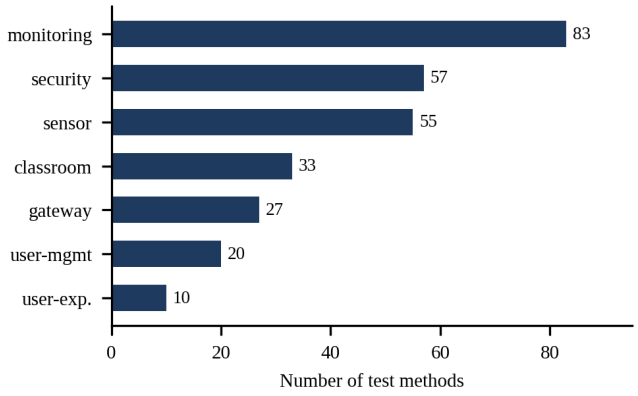
The source-code inventory contains 285 test methods in 42 classes (Table 3 and Figure 2). According to their annotations and base classes, 128 are unit tests with neither a Spring context nor containers, 89 use the Spring context (controllers and integration tests) and 68 depend on Docker containers (Testcontainers). They cover application services, controllers, JWT, the Redis revocation list, rate limiting, event consumers and the alert and analysis rules.

Table 3
Automated Tests per Service

Service	Classes	Methods
ms-environment-monitoring	12	83
ms-security	8	57
ms-sensor-management	7	55
ms-classroom-management	4	33
API Gateway	6	27
ms-user-management	2	20
ms-user-experience	3	10
Total	**42**	**285**

Note. Count taken from the source code, not from an execution.

Figure 2
Test Methods per Service



Note. Authors' own work based on the code inventory.

Manual Tests

The 21 web, mobile and IoT cases are manual, because the frontend has no automated tests. They verify role-based login, dashboard navigation, favorites, BLE provisioning, sensor readings and behavior under failure (disconnected sensor, revoked permissions, lost Wi-Fi).

Acceptance Criteria

The plan sets acceptance criteria for a local demonstration (Table 4). They are not a production service-level agreement.

Table 4
Test Plan Acceptance Criteria

Indicator	Criterion	Measurement
Health check	Under 1 s	Pending
Login	Under 2 s	Pending
Dashboard with test data	Under 3 s	Pending
HTTP 5xx errors	0 in the critical flow	Pending
Availability	100% of planned time	Pending

Note. Pending: the measurement has not been run yet.

Results

Results are presented as they stand on October 10, 2026. No performance metrics are reported because they have not been measured yet.

Deliverables

The outcome is a platform with seven backend modules, a web application and a mobile application covering the eight capabilities of the minimum scope: authentication, environments, sensors, monitoring, dashboard, historical analysis, favorites and environment ratings. Complete documentation was also delivered (requirements, analysis, technical proposal, design, database, test plan, quality, implementation, and technical, installation, administrator and user manuals), along with a physical prototype that demonstrates temperature, humidity and sound readings and BLE provisioning.

Test Execution Status

The execution record of October 9, 2026 is a draft: of the 48 cases, 47 are NOT RUN and one (IOT-008, sending measurements to the API) is BLOCKED because the sending firmware does not exist. As background, the execution reports of October 7 appear in Table 5. In ms-environment-monitoring, 58 of 86 tests failed because of a duplicated bean in the Spring context; only one class with that name exists in the source code, but the build directory keeps an outdated class, so a clean build will probably resolve it. If the error persists, it will be handled as a defect.

Table 5
Previous Execution Reports (October 7, 2026)

Service	Tests	Passed	With error
ms-user-experience	10	10	0
ms-environment-monitoring	86	28	58

Note. These are not the final outcome of the test round.

Risk Register

The code audit of October 8, 2026 left 11 active risks: 2 critical, 4 high and 5 medium. The critical ones are the possibility of impersonating an identity through the X-User-Id and X-User-Role headers when a service is reached directly, and a development private key committed to the repository together with a superadmin account seeded with a default password. The high ones include open CORS with credentials, the absence of coverage measurement, a service missing from Kubernetes and continuous integration, and observability that is only declared. Four other risks from the same audit are already closed, for example the token-signature incompatibility between services and an outbox that never published.

Discussion

Interpretation

Splitting the system into services, OpenAPI contracts and Liquibase migrations make it possible to evolve each module without breaking the others, which agrees with Fowler and Lewis (2014). The closed risks also show a pattern: features that lacked an integration test (the outbox,

the event chain, live metrics) failed without anyone noticing, which supports the importance that Martin (2008) and Forsgren et al. (2018) place on testing and continuous delivery.

Lessons Learned

Documenting and auditing honestly is as valuable as building: separating what has been executed from what is pending makes results verifiable and turns limitations into conscious decisions. It was also learned that documented decisions can drift from what is deployed, as happened with the database strategy: the ADR called for one instance with a schema per service, while Compose and Kubernetes start independent servers.

Threats to Validity

The work has three main threats. First, quantitative performance and availability results do not exist yet, so it cannot be claimed that the system meets the acceptance criteria. Second, the prototype was tested with controlled readings and not in a real classroom deployment. Third, part of the test inventory is a static count of the code and does not guarantee that the tests pass.

Conclusions

Summary of Findings

EduAirControl shows that it is feasible to centralize the environmental data of educational spaces in a secure, traceable and scalable platform, using hexagonal microservices and a low-cost IoT prototype. Its quality, however, is not yet demonstrated by measurements: what exists is a verifiable foundation (plan, cases, test inventory and risk register) ready to be executed.

Contributions

As a product, a working system with web, mobile and a physical prototype is delivered. As a technical contribution, a reference architecture is documented with 17 justified decisions and 53 contracted paths. As a methodological contribution, a test plan is presented with a results scheme that prevents recording as passed what has not been executed.

Limitations

The final firmware that sends measurements to the API does not exist yet. The MQ-135 is not calibrated. There are no automated frontend tests. Load testing, certified penetration testing and disaster recovery were not carried out. The two critical security risks remain open.

Future Work

In the short term: execute and close the test plan, fix the critical security risks, measure coverage with JaCoCo and complete the ingestion firmware with Wi-Fi reconnection. In the medium term: calibrate the MQ-135 or replace it with a certified CO_2 sensor, add automated web and mobile tests, implement the circuit breaker and observability, and deploy to a staging environment to validate rollback. In the long term: evaluate the system in real classrooms over an extended period and study predictive alerts.

Final Reflection

A measurable system starts by saying clearly what has been measured and what has not. That is the conclusion of this project: quality is not declared, it is demonstrated, and in the meantime what is missing must be written down.

References

Atzori, L., Iera, A., & Morabito, G. (2010). The Internet of Things: A survey. *Computer Networks, 54*(15), 2787–2805. https://doi.org/10.1016/j.comnet.2010.05.010

Cockburn, A. (2005). *Hexagonal architecture*. https://alistair.cockburn.us/hexagonal-architecture/

EduAirControl Team. (2026a). *EduAirControl technical manual* (Version 1.1) [Technical document]. SENA Industria.

EduAirControl Team. (2026b). *EduAirControl test plan* (Version 1.0) [Technical document]. SENA Industria.

EduAirControl Team. (2026c). *Project risk register* [Technical document]. SENA Industria.

Espressif Systems. (n.d.). *ESP32 series datasheet*. https://www.espressif.com/

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and DevOps*. IT Revolution Press.

Fowler, M., & Lewis, J. (2014). *Microservices: A definition of this new architectural term*. https://martinfowler.com/articles/microservices.html

Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M. (2013). Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems, 29*(7), 1645–1660. https://doi.org/10.1016/j.future.2013.01.010

Hardt, D. (Ed.). (2012). *The OAuth 2.0 authorization framework* (RFC 6749). Internet Engineering Task Force. https://doi.org/10.17487/RFC6749

International Organization for Standardization & International Electrotechnical Commission. (2011). *Systems and software engineering—Systems and software quality requirements and evaluation (SQuaRE)—System and software quality models* (ISO/IEC 25010:2011).

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.

Mendell, M. J., & Heath, G. A. (2005). Do indoor pollutants and thermal conditions in schools influence student performance?

A critical review of the literature. *Indoor Air*, *15*(1), 27–52. https://doi.org/10.1111/j.1600-0668.2004.00320.x

Newman, S. (2015). *Building microservices: Designing fine-grained systems*. O'Reilly Media.

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2nd ed.). Pragmatic Bookshelf.

OpenAPI Initiative. (2021). *OpenAPI specification* (Version 3.1.0). https://spec.openapis.org/oas/v3.1.0

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.

Shield, B. M., & Dockrell, J. E. (2003). The effects of noise on children at school: A review. *Building Acoustics*, *10*(2), 97–116.

Appendix

Data: figures come from the project source code (test inventory), the execution reports of October 7, 2026 and the risk register. No real personal data was used.

Code: project repository, listed in the file 15_Enlace_Repositorio_GitHub.txt of the delivery; version (commit) to be recorded.

Environment: Windows; earlier reports show JDK 25.0.2, whereas the POMs declare Java 17. Docker and Node versions to be recorded.

Annexes: OpenAPI contracts, ADRs, infrastructure files, risk register, screenshots and test inventory (folder 14_Anexos).